

Jobsheet Praktikum: Pertemuan 2

Kelas dan Objek

Mata Kuliah	Praktikum Pemrograman Berbasis Objek (RTI253008)
Pertemuan	2 (Minggu 2)
Durasi	1 × 4 × 50' praktikum; 1 × 1 × 50' tugas/laporan mandiri

A. Capaian Praktikum

Setelah menyelesaikan jobsheet ini, mahasiswa mampu:

1. Mendefinisikan kelas Java dengan atribut dan method, serta membuat objek darinya menggunakan `new`.
2. Menulis constructor untuk menjamin objek selalu lengkap datanya sejak awal dibuat.
3. Menulis method yang mengembalikan nilai dan menerapkannya untuk logika sederhana pada data sebuah objek.
4. Menjelaskan perilaku referensi (aliasing dan `null`) serta membuat banyak objek sekaligus lewat array.

B. Persiapan dan Prasyarat

- **Alat:** JDK 17 atau lebih baru, NetBeans (editor yang digunakan sepanjang praktikum ini).
- **Proyek:** pertemuan ini melanjutkan proyek bank-mini yang dibuat pada Pertemuan 1, dan langsung menerapkan teori kelas serta objek yang telah dibahas di kelas konsep pada Account, kelas pertama aplikasi Bank Mini. Seluruh kelas tetap ditempatkan dalam package `id.ac.polinema` (konvensi umum penamaan package di Java: nama domain institusi dibalik urutannya, “`polinema.ac.id`” menjadi `id.ac.polinema`).
- **Verifikasi cepat** sebelum memulai:

```
java -version
javac -version
```

Apabila keduanya menampilkan nomor versi tanpa galat, proses dapat dilanjutkan.

Tanpa NetBeans? Jobsheet ini tetap dapat diikuti menggunakan editor teks biasa:

```
javac -d out src/id/ac/polinema/*.java
java -cp out id.ac.polinema.Main
```

Checkpoint dan output yang dihasilkan tetap sama persis, apa pun editor yang digunakan.

C. Langkah Kerja

Langkah 1: Melanjutkan Proyek bank-mini

Buka proyek bank-mini yang dibuat pada Pertemuan 1 (atau folder yang sama bila mengikuti alternatif “Tanpa NetBeans?” di atas). Pertemuan ini membangun kelas Account, cikal bakal aplikasi Bank Mini, langkah demi langkah: mulai dari atribut saja, lalu method, lalu perilaku referensi, sampai akhirnya banyak Account dikelola sekaligus lewat array.

✓ **Checkpoint:** panel Projects masih menampilkan proyek bank-mini dengan package `id.ac.polinema` berisi `Main.java` dari Pertemuan 1.

Langkah 2: Kelas Account Minimal dan Objek Pertama

Kelas merupakan cetakan yang menjelaskan atribut dan perilaku yang akan dimiliki oleh setiap **objek** yang dibuat darinya. Kelas Account berikut baru memiliki atribut (state), belum memiliki method:

```

1 package id.ac.polinema;
2
3 public class Account {
4     public String ownerName;
5     public double balance;
6 }

```

Figure 1: Account.java: kelas dengan dua atribut

Tambahkan kelas baru Account ke package id.ac.polinema (klik kanan package > New > Java Class), lalu ganti isi Main.java untuk membuat satu objek Account dan mengisi atributnya satu per satu:

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc = new Account();
6         acc.ownerName = "Nadia";
7         acc.balance = 500000;
8         System.out.println(acc.ownerName + " - balance: " + acc.balance);
9     }
10 }

```

Figure 2: Main.java: membuat objek Account dan mengisi atribut secara manual

Jalankan proyek (klik kanan proyek > Run, atau tekan tombol F6). Apabila menggunakan terminal: `javac -d out src/id/ac/polinema/*.java && java -cp out id.ac.polinema.Main`.

✓ **Checkpoint:** program menampilkan Nadia - balance: 500000.0.

⚠ **Jika gagal:** apabila muncul error: cannot find symbol, periksa apakah nama berkas sama persis dengan nama kelasnya (Account.java untuk class Account). Java bersifat case-sensitive, sehingga huruf besar dan kecil harus sesuai.

Pada saat pernyataan `new Account()` dijalankan, terdapat dua hal yang terjadi di memori. Variabel `acc` hanyalah tempat kecil di **stack** yang berisi alamat. Objek Account yang sesungguhnya, lengkap dengan `ownerName` dan `balance`-nya, berada secara terpisah di **heap**. Ilustrasinya adalah sebagai berikut:

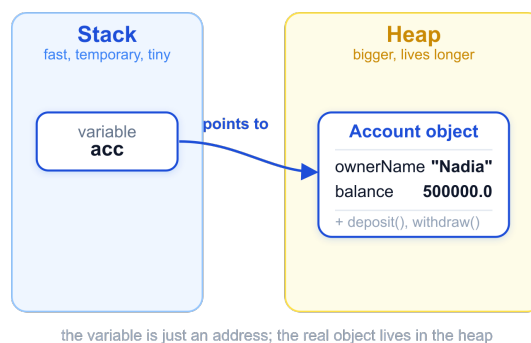


Figure 3: Variabel di stack menunjuk ke objek Account di heap

Langkah 3: Menambahkan Method (Perilaku)

Atribut hanya menyimpan data dan tidak memiliki perilaku. **Method** memberikan perilaku pada kelas, yaitu sesuatu yang dapat dilakukan objek dengan datanya sendiri. Tambahkan tiga method ke Account (baris hijau menandai kode yang baru ditambahkan):

Sederhanakan Main.java agar menggunakan ketiga method tersebut:

```

1 package id.ac.polinema;
2
3 public class Account {
4     public String ownerName;
5     public double balance;
6
7     public void deposit(double amount) {
8         balance = balance + amount;
9     }
10
11     public void withdraw(double amount) {
12         balance = balance - amount;
13     }
14
15     public void printInfo() {
16         System.out.println(ownerName + " - balance: " + balance);
17     }
18 }

```

Figure 4: Account.java dengan method deposit(), withdraw(), dan printInfo() ditambahkan

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc = new Account();
6         acc.ownerName = "Nadia";
7         acc.deposit(500000);
8         acc.withdraw(150000);
9         acc.printInfo();
10    }
11 }

```

Figure 5: Main.java memakai deposit(), withdraw(), dan printInfo()

Jalankan ulang.

✓ **Checkpoint:** program menampilkan Nadia - balance: 350000.0.

⚠ **Jika gagal:** apabila muncul galat non-static method deposit(double) cannot be referenced from a static context, method dipanggil tanpa melalui objek (misalnya Account.deposit(500000)). Method non-static wajib dipanggil melalui objeknya, yaitu acc.deposit(500000).

Diagram kelas UML berikut merangkum Account sejauh ini:

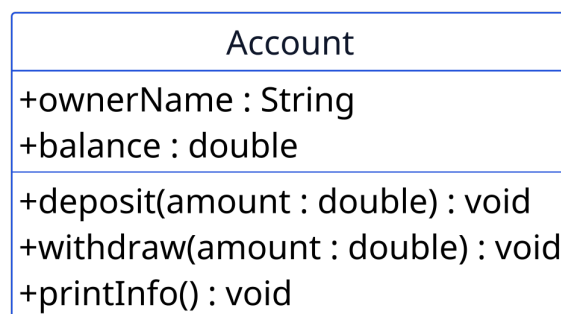


Figure 6: Diagram kelas UML untuk Account

Tanda + di depan atribut atau method menunjukkan sifat publik (dapat diakses langsung dari luar kelas), sedangkan tanda - menunjukkan sifat privat. Diagram di atas seluruhnya bertanda +, artinya ownerName dan balance masih dapat diubah langsung dari luar kelas, tanpa lewat deposit()/withdraw(). Ini disengaja untuk pertemuan ini; risiko dari desain seperti ini menjadi alasan munculnya encapsulation, yang dibahas tuntas pada Pertemuan 3.

Langkah 4: Konstruktor

Perhatikan risiko berikut pada Main.java sejauh ini: apabila baris `acc.ownerName = "Nadia";` pada Langkah 2 tidak sengaja terlewat sebelum `printInfo()` dipanggil, program tetap berjalan tanpa galat, tetapi mencetak `null - balance: 0.0`, sebuah bug yang mudah terlewat karena tidak ada exception yang menghentikan program. **Konstruktor** menutup celah ini dengan mewajibkan data lengkap pada saat objek dibuat:

```
1 package id.ac.polinema;
2
3 public class Account {
4     public String ownerName;
5     public double balance;
6
7     public Account(String ownerName, double balance) {
8         this.ownerName = ownerName;
9         this.balance = balance;
10    }
11
12    public void deposit(double amount) {
13        balance = balance + amount;
14    }
15
16    public void withdraw(double amount) {
17        balance = balance - amount;
18    }
19
20    public void printInfo() {
21        System.out.println(ownerName + " - balance: " + balance);
22    }
23 }
```

Figure 7: Account.java dengan constructor ditambahkan

`this.ownerName` merujuk pada atribut milik objek, sedangkan `ownerName` di sisi kanan merupakan parameter konstruktor. Karena kedua nama tersebut sengaja dibuat sama, kata kunci `this` diperlukan agar Java dapat membedakan keduanya. Sederhanakan Main.java agar memakai konstruktor ini:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc = new Account("Nadia", 500000);
6         acc.withdraw(150000);
7         acc.printInfo();
8     }
9 }
```

Figure 8: Main.java memakai konstruktor Account

✓ **Checkpoint:** program menampilkan `Nadia - balance: 350000.0`, output sama persis dengan Langkah 3, kode Main.java kini jauh lebih ringkas.

⚠ **Jika gagal:** apabila muncul galat `constructor Account in class Account cannot be applied to given types`, periksa apakah jumlah dan urutan argumen pada `new Account(...)` sudah sesuai dengan parameter konstruktornya.

Langkah 5: Method dengan Logika dan Nilai Kembali

Method tidak harus bertipe `void`. Method yang mengembalikan nilai memproses data objek lalu menyerahkan hasilnya kepada pemanggil lewat `return`. Tambahkan dua method berikut ke Account, sekaligus perbarui `withdraw()` agar memanfaatkan salah satunya:

`formatBalance()` mengubah `balance` menjadi teks dengan pemisah ribuan dan dua angka desimal lewat `String.format("%.2f", balance)`. `isOverdrawn()` mengembalikan `true` apabila saldo sudah negatif, dan `withdraw()` kini memanfaatkannya: mengurangi saldo lebih dulu, memeriksa lewat `isOverdrawn()`, lalu membatalkan pengurangan tersebut (mengembalikan saldo seperti semula) apabila ternyata membuat saldo negatif. Perbarui Main.java untuk mencoba semuanya, termasuk sengaja menarik saldo melebihi batas yang tersedia:

```

1 package id.ac.polinema;
2
3 public class Account {
4     public String ownerName;
5     public double balance;
6
7     public Account(String ownerName, double balance) {
8         this.ownerName = ownerName;
9         this.balance = balance;
10    }
11
12    public void deposit(double amount) {
13        balance = balance + amount;
14    }
15
16    public void withdraw(double amount) {
17        balance = balance - amount;
18        if (isOverdrawn()) {
19            balance = balance + amount;
20            System.out.println("Withdrawal rejected: insufficient balance.");
21        }
22    }
23
24    public void printInfo() {
25        System.out.println(ownerName + " - balance: " + balance);
26    }
27
28    public String formatBalance() {
29        return String.format("%.2f", balance);
30    }
31
32    public boolean isOverdrawn() {
33        return balance < 0;
34    }
35 }

```

Figure 9: Account.java dengan method formatBalance() dan isOverdrawn() ditambahkan, withdraw() diperbarui

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account acc = new Account("Nadia", 500000);
6         acc.withdraw(150000);
7         acc.printInfo();
8         System.out.println("Formatted: " + acc.formatBalance());
9
10        acc.withdraw(1000000);
11        acc.printInfo();
12    }
13 }

```

Figure 10: Main.java mencetak saldo terformat, lalu menguji penarikan yang ditolak

✓ **Checkpoint:** program mencetak empat baris: Nadia - balance: 350000.0, Formatted: 350,000.00, Withdrawal rejected: insufficient balance., lalu Nadia - balance: 350000.0 lagi (saldo kembali seperti semula karena penarikan kedua ditolak).

⚠ **Jika gagal:** apabila hasil `formatBalance()` tidak menampilkan pemisah ribuan, periksa kembali format string `"%,.2f"`: tanda koma sebelum `.2f` yang mengaktifkan pemisah ribuan.

Catatan. Cara `withdraw()` menolak penarikan di atas cukup janggal: mengurangi saldo dulu, memeriksa lewat `isOverdrawn()`, baru membatalkan bila ternyata salah. Cara yang lebih bersih adalah memeriksa kecukupan saldo SEBELUM saldo diubah sama sekali, itulah yang dilakukan Pertemuan 3. Lebih penting lagi, validasi ini bisa dilewati sepenuhnya: karena `balance` masih atribut publik, kode lain bebas menulis `acc.balance = -999999`; secara langsung, tanpa pernah melalui `withdraw()` atau `isOverdrawn()` sama sekali. Encapsulation pada Pertemuan 3 menutup celah ini: begitu `balance` menjadi privat, satu-satunya jalan mengubahnya adalah lewat method yang sudah divalidasi.

Langkah 6: Referensi, Aliasing, dan null

Variabel objek di Java bukan merupakan objeknya sendiri, melainkan **referensi** yang menunjuk ke objek di memori. Oleh karena itu, dua variabel dapat menunjuk ke objek yang persis sama. Ganti isi `Main.java` dengan kode berikut:

```
1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account original = new Account("Nadia", 500000);
6         Account copy = original;
7         copy.deposit(100000);
8
9         System.out.println("Via original: " + original.balance);
10        System.out.println("Via copy: " + copy.balance);
11
12        Account empty = null;
13        empty.printInfo();
14    }
15 }
```

Figure 11: `Main.java` dengan blok aliasing dan uji null ditambahkan

Jalankan.

✓ **Checkpoint:** kedua baris pertama (Via original: dan Via copy:) sama-sama menampilkan nilai 600000.0, meskipun hanya `copy` yang menerima deposit kedua. Hal ini bukan merupakan galat, melainkan cara kerja referensi: `original` dan `copy` menunjuk ke objek yang sama persis, sehingga perubahan melalui salah satu variabel otomatis terlihat pada variabel lainnya. Baris berikutnya akan menghasilkan `NullPointerException`, dan hal ini memang disengaja.

Berikut ilustrasinya pada stack dan heap:

Perbaiki dengan menghapus baris terakhir (`Account empty = null;` beserta pemanggilan `printInfo()` di atasnya) agar program dapat berjalan kembali tanpa galat:

⚠ **Jika gagal:** `NullPointerException` selalu muncul apabila method dipanggil pada referensi yang belum menunjuk ke objek mana pun (`null`). Solusinya selalu sama, yaitu memastikan objek telah benar-benar dibuat dengan `new` sebelum method-nya dipanggil.

Langkah 7: Array of Objects, Banyak Objek dari Satu Kelas

Satu kelas dapat menghasilkan banyak objek sekaligus. Ganti isi `Main.java` dengan array `Account[]` berisi tiga rekening:

✓ **Checkpoint:** program mencetak tiga baris - balance: (satu per elemen array, dengan nilai yang berbeda-beda karena setiap `Account` memiliki datanya sendiri): Nadia - balance: 350000.0, Budi - balance: 1000000.0, Sari - balance: 500000.0.

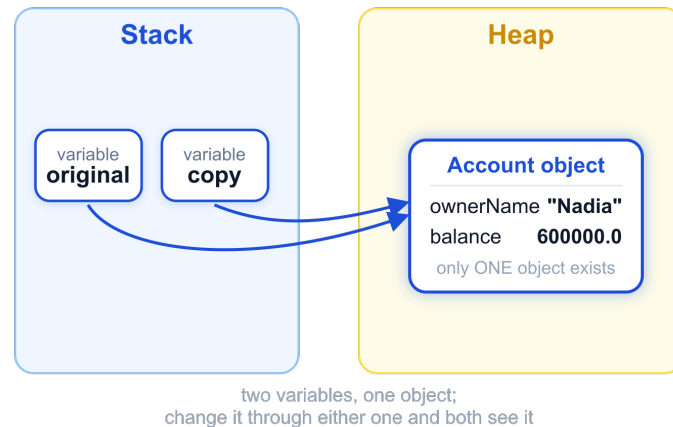


Figure 12: Dua variabel di stack menunjuk ke satu objek Account yang sama di heap

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account original = new Account("Nadia", 500000);
6         Account copy = original;
7         copy.deposit(100000);
8
9         System.out.println("Via original: " + original.balance);
10        System.out.println("Via copy: " + copy.balance);
11    }
12 }

```

Figure 13: Main.java setelah baris uji null dihapus

```

1 package id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Account[] accounts = new Account[3];
6
7         accounts[0] = new Account("Nadia", 500000);
8         accounts[0].withdraw(150000);
9
10        accounts[1] = new Account("Budi", 1000000);
11
12        accounts[2] = new Account("Sari", 750000);
13        accounts[2].withdraw(250000);
14
15        for (Account acc : accounts) {
16            acc.printInfo();
17        }
18    }
19 }

```

Figure 14: Main.java final: array Account[] berisi tiga rekening

⚠ **Jika gagal:** `ArrayIndexOutOfBoundsException` menunjukkan bahwa indeks yang diisi tidak tersedia, misalnya `accounts[3]` padahal array tersebut hanya berukuran 3 (indeks yang valid: 0, 1, 2).

D. Tugas dan Deliverable

Kumpulkan hal berikut sesuai format yang diminta Dosen:

- Screenshot output program setelah Langkah 7.
- **Tugas mandiri:**
 1. Bank Mini perlu memindahkan saldo antar rekening. Tambahkan method `transferTo` ke `Account` sesuai diagram kelas UML berikut. Tidak ada contoh kode untuk langkah ini, rancang sendiri isinya berdasarkan method yang sudah tersedia (`deposit()`, `withdraw()`) pada diagram:

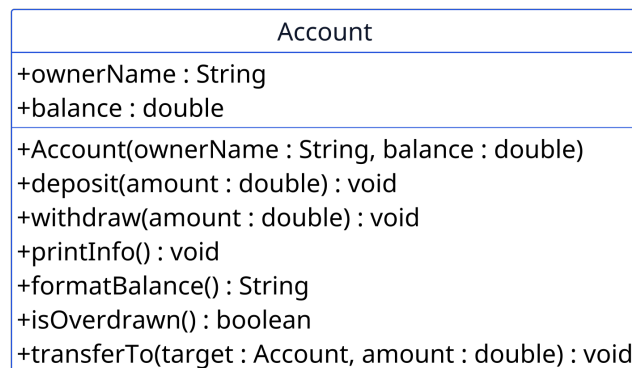


Figure 15: Diagram kelas UML untuk `Account` dengan tambahan `transferTo`

Buktikan dengan membuat dua objek `Account` di `Main`, memanggil `transferTo` dari salah satunya ke yang lain, lalu mencetak keduanya:

```
1 package id.ac.polinema;  
2  
3 public class Main {  
4     public static void main(String[] args) {  
5         Account from = new Account("Nadia", 500000);  
6         Account to = new Account("Budi", 200000);  
7  
8         from.transferTo(to, 100000);  
9  
10        from.printInfo();  
11        to.printInfo();  
12    }  
13 }
```

Figure 16: `Main.java` menguji `transferTo` antar dua `Account`

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan): (a) apa perbedaan antara objek dengan referensi ke objek? (b) kapan tepatnya konstruktor sebuah kelas dijalankan?

E. Kriteria Penilaian

Komponen	Bobot	Kriteria Lengkap (100%)	Kriteria Minimum
Langkah kerja tuntas	40%	Seluruh langkah dijalankan dan berfungsi	Sebagian besar langkah selesai, hasil akhir berjalan
Checkpoint terverifikasi	35%	Semua checkpoint tercapai dan dibuktikan (screenshot/output)	Sebagian checkpoint terbukti
Tugas mandiri	25%	Method transfer antar rekening benar dan jawaban konsep tepat	Method transfer antar rekening ada meski jawaban belum lengkap